Milestone 1 · Oct 7, 2026, 12:00 (Beijing time)
第一阶段 · 2026年10月7日中午12:00（北京时间）
Submissions received by then will inform selection of collaboration proposals before the college project application deadline on Oct 10. / 届时收到的提交将用于遴选合作提议，以便在10月10日学院立项截止前推进。

Milestone 2 · Oct 12, 2026, 12:00 (Beijing time)
第二阶段 · 2026年10月12日中午12:00（北京时间）
Final homework submission deadline. / 作业最终提交截止时间。

02 / ASSIGNMENT

Assignment statement

Exact workload, design space, assembly rules, scoring, and submission format.

▶ Browse all resources

1. The challenge

Submit **one accelerator configuration and two programs**. The same hardware must run both scenarios: prompt prefill plus its first new position, and a sequence of decode positions. Your goal is to produce correct hidden states and KV updates while improving the combined simulated score. You may change hardware choices, tiling, data movement, workgroup placement, scheduling, fusion, recomputation, attention implementation, and the assembly text.

This is a single-chip, FP32, decoder-only Transformer inference problem. In each layer, an incoming hidden state passes through affine LayerNorm, a QKV projection, dense causal attention, an output projection and residual, a second LayerNorm, then `w1 + b1`, tanh-approximate GELU, `w2 + b2`, and a second residual. Q, K, and V are split into heads; each layer writes its newly computed K/V. The final layer writes the hidden state. There is no training, token sampling, vocabulary projection, MoE, quantization, or cross-chip communication. The public reference implementation defines the precise computation.

Workload supplied to your programs

Property	Both scenarios
Models	P1: three layers, width 256, eight heads of width 32, FFN width 1024 (about 2.37M parameters). D1: two layers, width 128, four heads of width 32, FFN width 512 (about 0.40M parameters). There is no vocabulary embedding or output head.
Data	FP32 weights, activations, and KV; LayerNorm epsilon `1e-5`
Attention	Dense causal self-attention with scores scaled by `1/sqrt(32)`
Output check	Final hidden states and every layer's new K/V, with `abs(candidate - reference) <= 1e-3 + 1e-3 * abs(reference)` ; nonfinite values fail

Case requirements

Case	What is ready at the start	What your program must do	Step boundary
`M1_P1`	Batch 2, with a 64-position prompt for each batch element	Compute both prompts and their K/V through three layers; then compute the first new position for each, attending to its own prompt and itself	Write **both** prompts' outputs/KV and issue `STEP.COMMIT {"step":0}` **before** reading either `input/step0` vector. Then write both 65th outputs and KV.
`M2_D1`	Batch 1, per-layer K/V for a 128-position history already in HBM	Compute eight consecutive new positions through the smaller two-layer model; step `s` attends to 128 historical plus `s+1` generated positions	Write step `s` 's output/KV and issue `STEP.COMMIT {"step":s}` for every `s=0..7` . Later external inputs are released only after earlier commits.

The new inputs are teacher-forced hidden-state vectors. You do not choose a vocabulary token. Preparing D1's 128-position history is outside its timed interval. The official numerical values are hidden, while shapes, addresses, and scoring rules are public.

Independent agent work

Use **your own AI agent** to design the hardware, produce the assembly, run experiments, and decide what to submit. Complete this work independently. Do not ask another person or another participant's agent for solutions, hints, code, designs, or feedback, and do not study or reuse their submissions or agent traces. The official starter package, this site's references, and publicly documented tools are allowed.

Every submission must include your own project and agent history to show how the result was reached.

How you search is your choice. Mixed-integer linear programming (MILP) is one possible way to model a bounded set of hardware and software decisions; **an MILP model is not required**. You may instead use an agent-guided search, heuristics, a compiler optimization loop, or a combination. The judge evaluates the submitted hardware and assembly, not the method used to find them.

---

2. Hardware menu and design space

The machine these choices describe

This is a **GPU-inspired teaching accelerator at a coarse architectural level**. One chip has identical streaming multiprocessors (SMs), a shared network-on-chip (NoC), high-bandwidth memory (HBM) channels, and an optional chip-wide cache. Each SM has tensor-core-like matrix units (TCs), vector lanes, special-function units (SFUs), zero to two dedicated reduction units, a fixed 256 KiB register file (RF), optional shared memory (SH), and direct-memory-access (DMA) engines for transfers. The chip has 2 GiB of addressable HBM. This model specifies the resources a program can use and the contention that affects its simulated time; it does not define CUDA threads, warps, a production GPU memory hierarchy, or a silicon implementation.

Your assembly creates workgroups with `WG.BEGIN` and names the destination SM explicitly. At most four workgroups can reside on one SM at a time. Each workgroup has a private 64 KiB RF slot and reserves its own SH quota from that SM. `LD` and `ST` move values among HBM, RF, and SH; `MMA.ACC`, `VEC`, `REDUCE`, and `SFU` compute from RF operands. RF `lane` numbers identify storage partitions, not GPU threads. A larger `sm_count` therefore helps only when your program places useful work on more SMs. The assembly reference defines these program-visible rules.

`hardware.json` must include `"version": "challenge-hardware-v0.4"` and all required menu fields. The baseline hardware JSON is a legal starting point. Units below are per SM unless a row says chip-wide.

Swipe across the table to read each field's explanation.

Field	Legal choices	Meaning in this model
`version`	`challenge-hardware-v0.4`	Hardware format identifier; required, not a performance choice.
`sm_count`	4, 8, 12, 16, 20, 24, 28, 32	Number of available SMs. More SMs cost area; programs must assign workgroups to use them.
`tc_count`	0, 1, 2, 3, 4, 6, 8	Matrix engines per SM for `MMA.ACC`; zero makes that opcode unavailable.
`tc_array` when TC > 0	`4x8`, `8x8`, `8x16`	Matrix engine's M-by-N array shape; changes throughput, tail utilization, and area.
`tc_k_parallel` when TC > 0	1, 2, 4	Parallelism along a matrix product's K reduction; changes cost and FP32 summation order.
`vector_lanes`	8, 16, 32, 64	Parallel lanes for `VEC`; also partitions each workgroup's fixed RF slot into this many lanes.
`sfu_lanes`	Any integer from 1 through `vector_lanes`	Lanes for `SFU` operations such as `exp`, `tanh`, and `rsqrt`.
`reduction_units`	0, 1, 2	Dedicated capacity for `REDUCE`; with zero, reductions use the vector path.
`rf_ports`	`2R1W`, `4R2W`, `8R4W`	Abstract RF read/write bandwidth; higher port counts cost more area and energy.
`shared_kib`	0, 32, 64, 96, 128, 192, 256	SH capacity per SM that resident workgroups may reserve; zero disables SH use.
`shared_banks` when shared > 0	1, 2, 4, 8	Number of independently serviced SH banks; accesses to one bank can contend.
`shared_ports` when shared > 0	`1R1W`, `2R1W`	Per-bank SH read/write service mode; more read bandwidth costs area.
`dma_depth`	0, 1, 2	Outstanding transfer slots per DMA engine; zero makes transfers block later issue by that workgroup.
`dma_engines`	1, 2, 4	Transfer engines per SM for `LD` and `ST`; more engines can serve transfers concurrently.
`sm_noc_bytes_per_cycle`	32, 64, 128	Byte capacity of each SM's links into and out of the NoC.

Field	Legal choices	Meaning in this model
`multicast`	`false` , `true`	Allows a concurrent HBM/cache line read to serve multiple destinations in the timing model.
`noc_bytes_per_cycle`	64, 128, 256, 512	Chip-wide NoC byte capacity shared by SM traffic.
`hbm_channels`	1, 2, 4, 8	Number of parallel HBM service channels; addresses are distributed across them.
`cache_mib`	0, 1, 2, 4, 8, 16	Capacity of the automatic chip-wide cache for HBM reads; zero bypasses it.

When `tc_count` is zero, **omit** `tc_array` and `tc_k_parallel` . When `shared_kib` is zero, **omit** `shared_banks` and `shared_ports` . The Cache is automatic shared hardware, with no assembly cache-control instruction. `multicast` is a JSON boolean.

Where the architectural rules live

The starter ZIP includes the full public implementation. Use the hardware menu and area formula and area, energy, clock, and budget coefficients for configuration costs. Functional instruction execution defines FP32 results and legal RF/SH/HBM accesses; the ISA reference explains that behavior in student-facing form. Compute service formulas and timed resource scheduling model issue, compute units, RF/SH ports, DMA, NoC, HBM contention, and transfers. Cache tags and traffic, timed cache service, and peak-power calculation complete the public cost path. The HBM ABI lists every workload symbol and address.

Functional correctness is checked by an FP32 instruction executor; timing is estimated by a deterministic resource scheduler. The area and energy coefficients are teaching estimates at a fixed 500 MHz clock, with no RTL synthesis or silicon measurement. Treat the resource behavior defined here as the machine for this assignment; familiar GPU design rules remain hypotheses to test against it.

The menu has about **120.7 billion raw hardware combinations**, even before choosing a program. Many fail area, power, or latency limits, and some may behave similarly; the count alone does not prove the search is hard. Evaluating all hardware/program pairs by brute force is not a useful strategy. Hardware value also depends on software: more Tensor Cores need suitable tiles and data supply, while a Cache or shared-memory allocation helps only when the access pattern actually reuses data. Familiar GPU rules

are useful hypotheses, but this machine has its own RF, memory, power, and cost model. Measure candidates in this simulator rather than assuming a GPU-derived choice wins here.

We deliberately restrict the model family, data type, hardware menu, and ISA. The challenge still includes enough interacting choices to require joint reasoning without asking you to design an unrestricted chip or compiler.

---

3. What to submit

Create one ZIP with these paths **at its root**. There is one submission type: each upload is graded, and your **highest eligible server score** under the same student ID appears on the leaderboard and counts as your final grade.

Milestone 1 / 第一阶段：October 7, 2026, 12:00 Beijing time / 2026年10月7日中午12:00（北京时间）。 Submissions received by this time will inform the selection of collaboration proposals before the college project application deadline on October 10. / 届时收到的提交将用于遴选合作提议，以便在10月10日学院立项截止前推进。 You may continue improving and submitting your homework afterward. / 此后仍可继续改进并提交作业。

Milestone 2 / 第二阶段：October 12, 2026, 12:00 Beijing time / 2026年10月12日中午12:00（北京时间）。 This is the final homework submission deadline; the server stops accepting new submissions then. Submissions received before the deadline may finish grading afterward. / 这是作业最终提交截止时间；届时服务器停止接收新提交。截止前收到的提交可在此后完成评分。

Required path	Contents
`hardware.json`	One legal configuration shared by P1 and D1
`programs/M1_P1.asm`	Complete prefill plus first-new-position program
`programs/M2_D1.asm`	Complete eight-step decode program
`project/iteration-log.md`	Your chronological project record: candidate changes, local checks, scores or observations, rejected ideas, and the reason for this submission

Required path	Contents
`agent_trace/`	Export the complete trace of the agent sessions used for this project in their native format, such as Codex session JSON/JSONL; keep at least one nonempty trace file under this folder

Keep the project files that produced your assembly, such as generator source, scripts, and experiment notes, under `project/` when you use them. Project records and agent traces are for **manual review**; the upload service does not automatically check whether they are present or complete. The grading service reads the three hardware/program files as data and does not execute submitted Python, shell scripts, or custom simulators. Each `.asm` source is limited to 8 MiB, 32 nested static loops, and 10 million instructions after expansion.

Additional submission information

1. **The submission system's score is authoritative.** It runs the provided default scorer against the current published assembly ISA. You may change the software optimization strategy and compiler, but the emitted assembly must remain compatible with that scorer. The scorer may contain bugs; if you find one, contact the TA at yiding@slai.edu.cn or on WeChat.
2. **Validate locally before uploading.** Automatic grading usually takes **10–30 minutes** and may take longer when many submissions are queued. Do not use the submission system as your validation loop. The **Submit** tab accepts one ZIP of at most **25 MiB**, saves it, and returns a receipt ID and lookup key. Keep both; **Submissions & lookup** shows progress and diagnostics. **Upload frequency: one submission per student ID every 10 minutes.** The server reports the remaining wait if you upload sooner. Please use the service considerately.
3. **Use a consistent identity.** Enter the same display name and your real student ID on every upload. Submissions are grouped by student ID for the leaderboard and final grade.

---

4. Resources and development workflow

Download the complete starter ZIP and extract it into a working directory. It is a **working baseline**, not just an empty template. It includes:

File or folder inside the ZIP	Purpose
`hardware.json`	Legal baseline hardware; use it directly or change menu values
`programs/M1_P1.asm`, `programs/M2_D1.asm`	Complete assembly programs for both model and scenario pairs; the unchanged baseline scores 1000
`project/compiler.py`	Editable Python assembly generator with `Builder`, `generate_m1_p1()`, `generate_m1_d1()`, and a command to write both `.asm` files
`codesign/challenge/baseline.py`	Frozen reference version of the generator used to verify the baseline manifest; leave this copy unchanged
`challenge.py` and the other files in `codesign/challenge/`	Public parser, functional checker, performance model, workload definition, HBM layout, and reference implementation
`baseline_manifest.json`	Frozen baseline cycles, hashes, and runtime contract used by local experimental scoring
`BACKGROUND.md`, `README.md`, `ASSEMBLY_GUIDE.md`, `ISA.md`, `ABI.md`	The motivation, this statement, a guided map of the long programs, complete instruction rules, and every public HBM address

Read these beside the code: assembly walkthrough · complete ISA · HBM ABI · full P1 listing · full D1 listing. P1 is about 401 KiB and D1 about 552 KiB because they spell out batch phases and decode steps; the walkthrough maps them by line range. Edit `project/compiler.py` and regenerate both files instead of hand-editing thousands of lines.

The supplied generator and your own compiler

The supplied `project/compiler.py` is a **small, editable Python compiler for this fixed workload**, copied from the reference generator. Its Python source describes the baseline LayerNorm, projections, attention, FFN, storage, and loop structure. It uses the public model/scenario definition and HBM layout, then emits literal ISA text for the two `programs/*.asm` files. It does **not** take a C program, run on input tensor values, or automatically choose a schedule from `hardware.json`; the checker supplies tensor values later and evaluates the generated assembly with the hardware file you submit. From the extracted starter directory, run `python -m project.compiler` to write both assembly files.

You may modify this generator or build a new one. A general-purpose programming language frontend is unnecessary: your generator only needs to produce legal assembly for the two fixed cases. It may implement your own software optimizations, including tile sizes and tail tiles, loop order, reuse in RF or shared memory, fusion, scratch allocation, workgroup placement, and synchronization. You may also add fast syntax, shape, resource, or dependency checks to reject bad candidates before the full public check. The published ISA and checker determine whether the emitted program is legal and correct. Keep the generator and any optimization scripts in `project/` for each submission; the grader executes only the emitted assembly.

Regenerate the unmodified baseline with `python -m project.compiler`. The assembly guide §7: Generate, check, and submit explains the compiler's inputs and outputs, its current tiling support, and the subsequent `check` and `estimate` commands.

A productive iteration looks like this:

1. **Reproduce the baseline.** Keep the provided files intact for one local `check` and `grade` run; inspect the case results and resource diagnostics.
2. **Change one idea at a time.** Start from the baseline generator or your own generator. If you change hardware, keep one `hardware.json` for both programs. Regenerate the `.asm` files you actually intend to submit.
3. **Check semantics before speed.** Run `check` on public seed 7. `estimate` is faster for performance exploration but does not establish correctness. Run `grade` for a public experimental score after correctness passes.
4. **Keep an iteration record.** Update `project/iteration-log.md` as candidates are tested. Export your own agent session trace to `agent-trace/`; include both in every ZIP.
5. **Package and upload.** Put the required paths at the ZIP root, save the receipt, and look up the official hidden-input result. Public results help development; the leaderboard and final grade use your best eligible server result.

Use **Python 3.12.13** and **NumPy 2.5.3** for frozen-baseline comparisons. From the extracted starter directory, use a fresh report filename for every run:

```
python challenge.py check --hardware hardware.json --program-p1 programs/M1_P1.asm --pr
python challenge.py estimate --hardware hardware.json --program-p1 programs/M1_P1.asm -
python challenge.py grade --hardware hardware.json --program-p1 programs/M1_P1.asm --pr
```

The cost coefficients, including area and power, are in `codesign/challenge/cost_v04.json`; the performance model source is in the same folder. The public `grade` result is **experimental**. Official hidden values and full reports are held on the server.

---

5. Assembly and storage rules

An assembly line is an opcode followed by one JSON object, such as `STEP.COMMIT {"step":0}`. Whole-line `#` comments are allowed. The complete ISA reference gives every opcode, field, vector operation, static-loop expression, and synchronization rule. The assembly walkthrough explains how the long example builds LayerNorm, QKV, attention, FFN, and output writes from those primitives.

Space	What an address means	Important rule	Full reference
HBM	**FP32 element offset**, not a byte address; the ABI table names each symbol, shape, offset, and release step	Weights and historical KV are read-only; future inputs are unreadable; output hidden/KV must be written to their published regions	HBM ABI
RF	Element offset within a workgroup's `lane` partition of its 64 KiB private slot	Initialize before reading; compute instructions take RF operands; another workgroup cannot read this slot	ISA operand format
SH	Element offset within explicitly allocated workgroup shared memory	Declare `shared_bytes` in `WG.BEGIN`; move SH values into RF before computing	ISA workgroup rules

For a row-major HBM matrix of width `N`, element `(row, column)` is at `base + row*N + column`. A strided tile needs `shape`, `strides`, and `count`: for a 48×16 tile from P1's `[256,768]` QKV weight matrix, use `shape:[48,16]`, `strides:[768,1]`, and `count:768`. The walkthrough's address example shows the full JSON view and K/V flattening formulas. A bare `count:768` reads consecutive elements and is a different tile.

- **Workgroups and compute.** `WG.BEGIN` / `WG.END` define lifetime. `LD` / `ST` move data. `MMA.ACC`, `VEC`, `REDUCE`, and `SFU` operate on RF. `FOR` / `END.FOR` are bounded static loops. The ISA opcode table lists exact fields.

- **Dependencies.** `WAIT` and `BARRIER` order named events. Source-text interleaving of different workgroups does not order overlapping HBM reads/writes. An `ST` may still be reading its RF/SH source after issue; wait before overwriting that source. See synchronization pitfalls.
- **Step release and coverage.** In P1, commit both complete prompts before reading the first new input. In D1, commit each of the eight outputs before the next input is available. Every required final hidden and new K/V element must be written exactly once. See P1/D1 program maps.
- **Numerics.** Instructions round to FP32 at defined boundaries. Stable softmax matters when logits are large. The hidden checker compares every required hidden/KV value with the tolerance above; a plausible runtime trace cannot substitute for correct values.

The fixed RF capacity, optional shared memory and Cache, and explicit movement rules make the hardware/software interaction measurable. You may use another valid program structure, including online softmax, as long as it follows this interface.

---

6. Scoring and leaderboard

The simulated clock is **500 MHz**. The frozen baseline takes **30,996,995 cycles** for P1 and **1,302,032 cycles** for D1. A submission is eligible only when both programs pass correctness, chip area is at most **24 mm²**, each case's maximum rolling **1000-cycle average power** is at most **20 W**, and each case takes no more than twice its own baseline cycles.

For eligible submissions:

```
score = 1000 × sqrt((30,996,995 / T_P1) × (1,302,032 / T_D1))
```

The unchanged v0.7 baseline scores **1000**. Earlier v0.6 receipts remain available for lookup, while the current leaderboard includes only v0.7 submissions because scores from different workloads cannot be compared. There is no preset maximum. A failed check or budget gate produces diagnostics and no leaderboard score. The leaderboard keeps each participant's best eligible official score. Receipt lookup shows case cycles, throughput, power, area, and gate results. These are estimates in the frozen educational simulator, not measured silicon PPA.

Agent-Guided Transformer Co-design